Introduction to Software Systems
COMP 206

Instructor: Jacob Errington & Max Kopinsky
School of Computer Science
McGill University

Lesson 1: First steps on the command line

Last time...

- Boring stuff: course outline.
- Interesting stuff: big picture of what a software system is

This time

- Preflights:

This time

- Preflights:
  - Computers

This time

- Preflights:
  - Computers
  - File System Tree and Paths

This time

- Preflights:
  - Computers
  - File System Tree and Paths
  - Basic FS Manip Commands

This time

- Preflights:
  - Computers
  - File System Tree and Paths
  - Basic FS Manip Commands
- Theory:

This time

- Preflights:
  - Computers
  - File System Tree and Paths
  - Basic FS Manip Commands
- Theory:
  - Communication among a system's components

This time

- Preflights:
  - Computers
  - File System Tree and Paths
  - Basic FS Manip Commands
- Theory:
  - Communication among a system's components
  - Deeper look: OS and role

This time

- Preflights:
  - Computers
  - File System Tree and Paths
  - Basic FS Manip Commands
- Theory:
  - Communication among a system's components
  - Deeper look: OS and role
  - Context and importance of Linux

This time

- Preflights:
  - Computers
  - File System Tree and Paths
  - Basic FS Manip Commands
- Theory:
  - Communication among a system's components
  - Deeper look: OS and role
  - Context and importance of Linux
  - Get started with Mimi

This time

- Preflights:
  - Computers
  - File System Tree and Paths
  - Basic FS Manip Commands
- Theory:
  - Communication among a system's components
  - Deeper look: OS and role
  - Context and importance of Linux
  - Get started with Mimi
- Practice: navigation on the command line

Recap: software system

A software system is a set of communicating software artifacts (programs), together with their configuration files, documentation, etc., that work together to achieve a shared goal.

- Communication is crucial, and takes different forms depending on the programs.
- Components are often written in different programming languages.
- A component of one system might itself be another system.

Example: enterprise expense reports

Example: enterprise expense reports

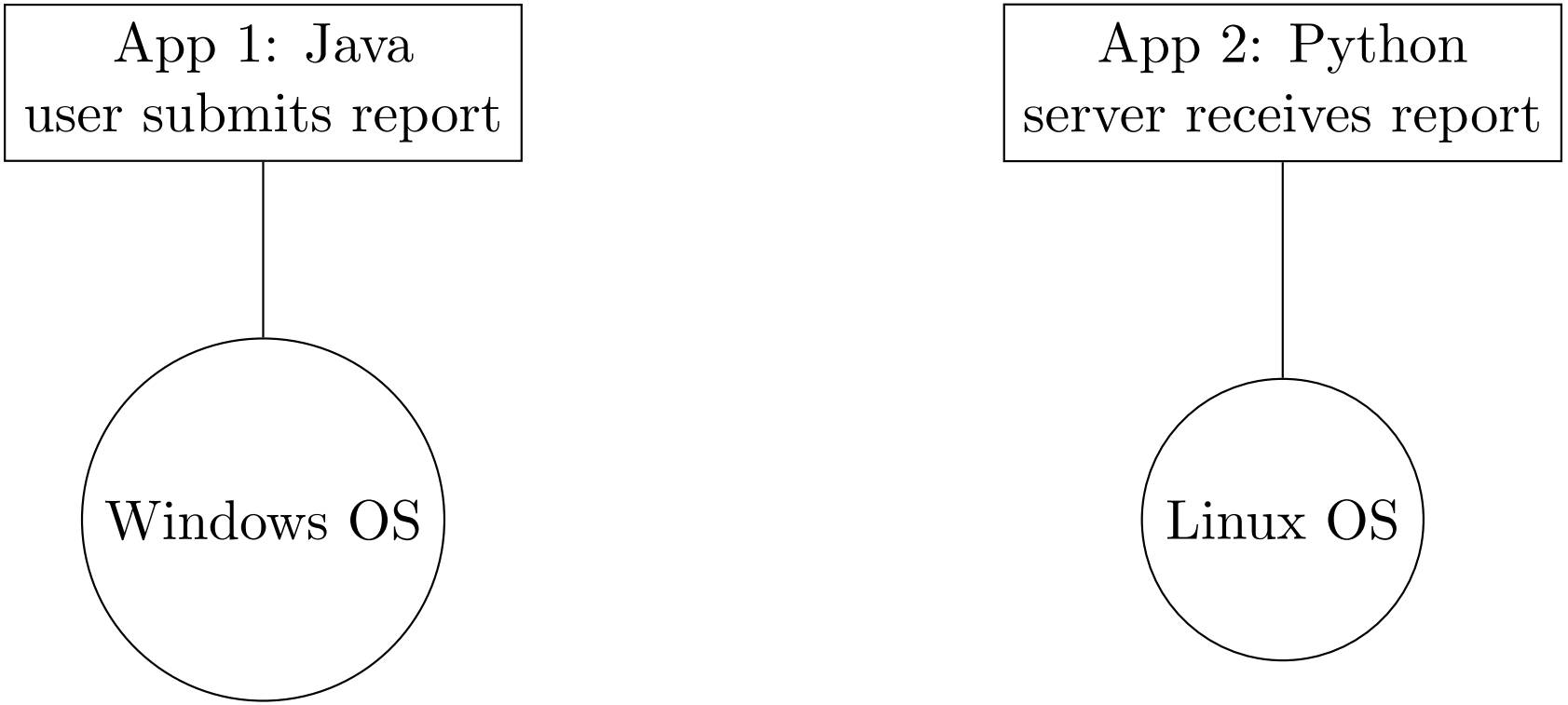
App 1: Java
user submits report

Example: enterprise expense reports

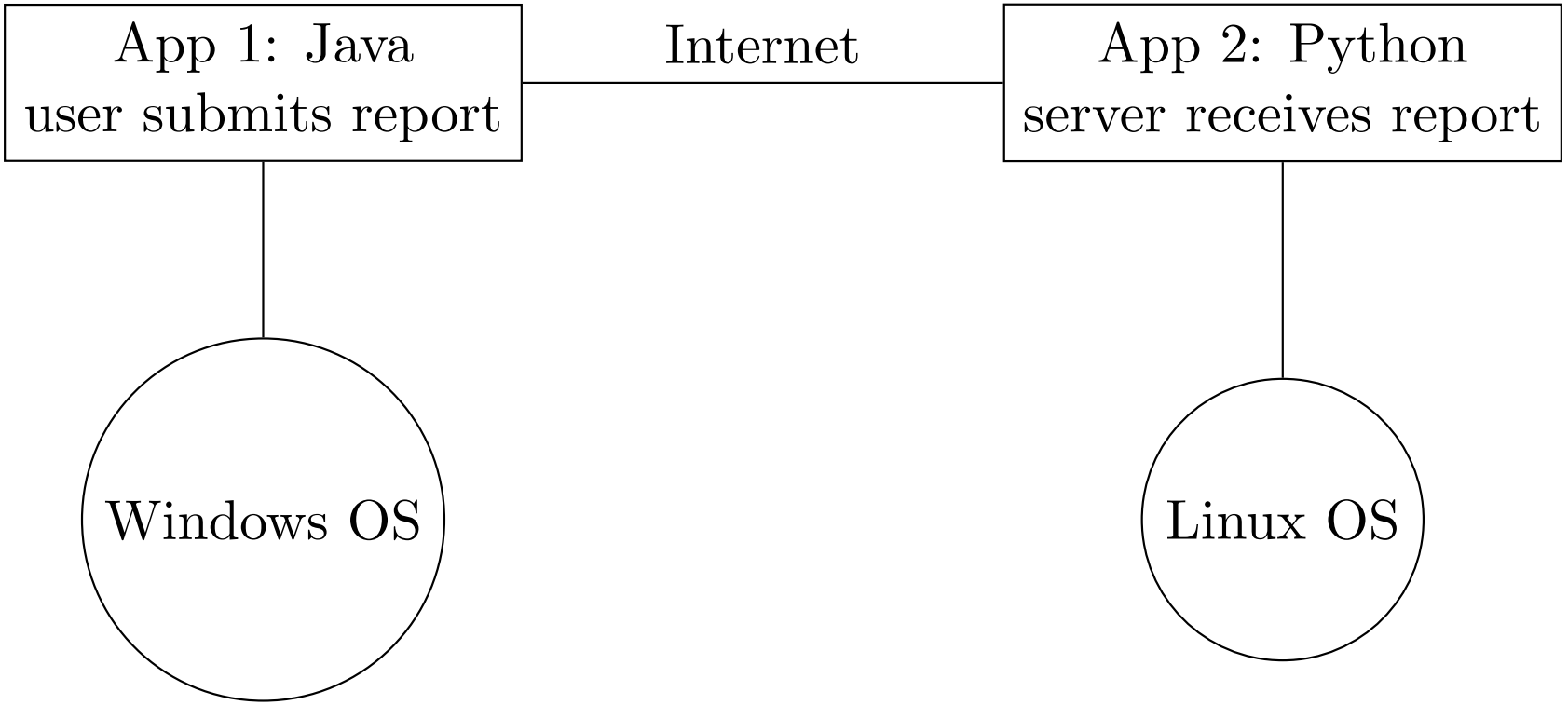
App 1: Java
user submits report

App 2: Python
server receives report

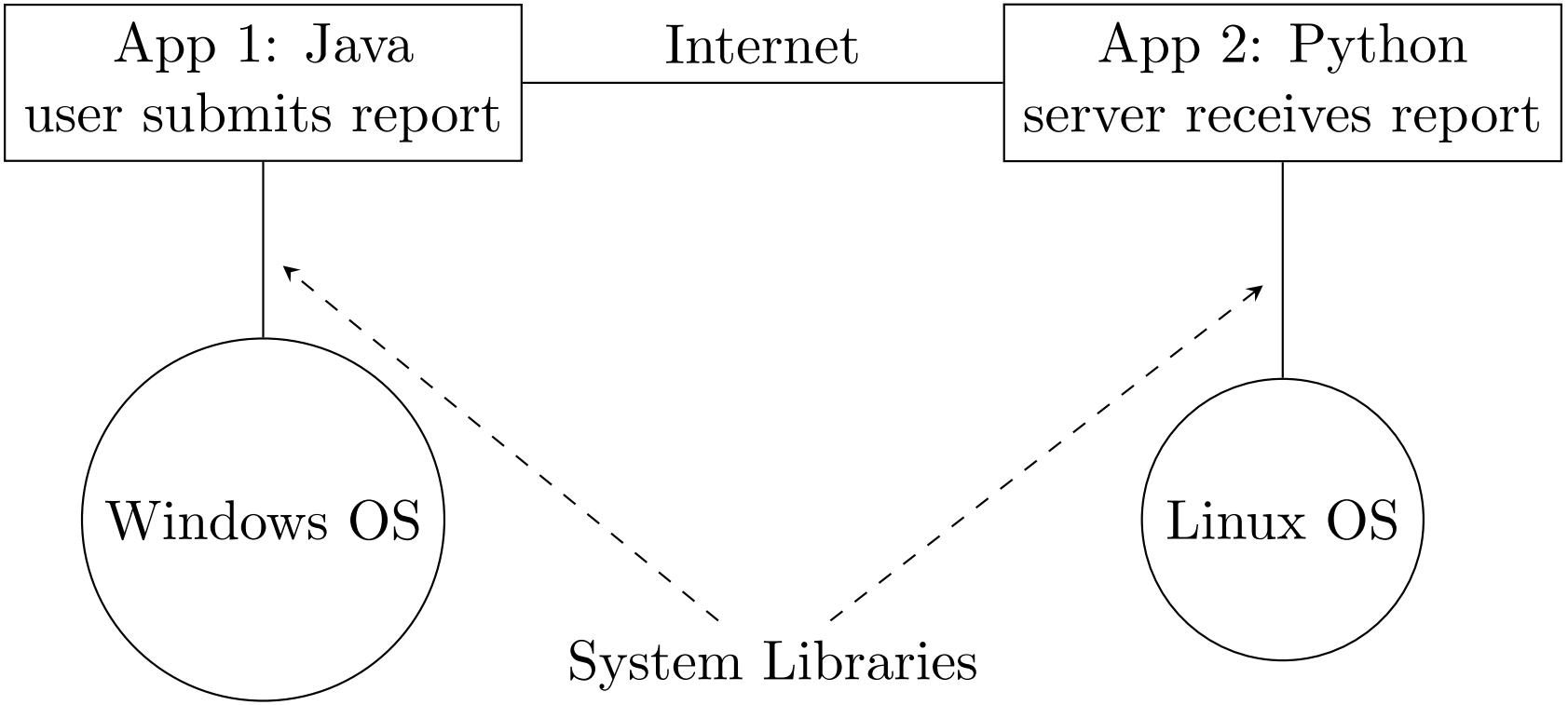
Example: enterprise expense reports



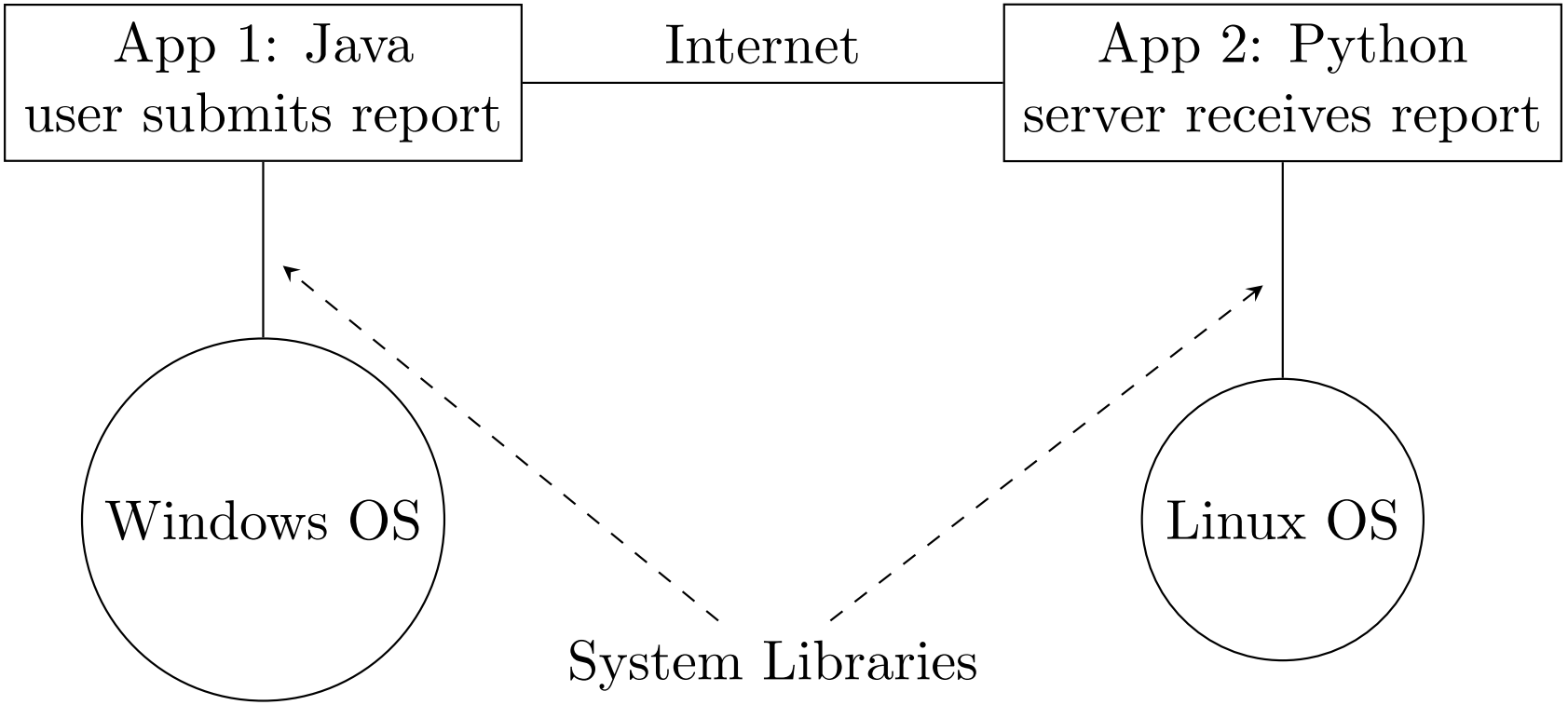
Example: enterprise expense reports



Example: enterprise expense reports



Example: enterprise expense reports



Think: what is the role of the OS here?

The role of the OS

- "Insulates" applications running on a computer, both from the *hardware* of that computer and also *from eachother*.

The role of the OS

- "Insulates" applications running on a computer, both from the *hardware* of that computer and also *from eachother*.
- "Insulates" the *user* from the complex inner workings of the computer.

The role of the OS

- "Insulates" applications running on a computer, both from the *hardware* of that computer and also *from eachother*.
- "Insulates" the *user* from the complex inner workings of the computer.
- OS is itself a system! Components include:
  the *kernel*, the network stack, the filesystem, a hardware abstraction layer

The role of the OS

- "Insulates" applications running on a computer, both from the *hardware* of that computer and also *from eachother*.
- "Insulates" the *user* from the complex inner workings of the computer.
- OS is itself a system! Components include:
  the *kernel*, the network stack, the filesystem, a hardware abstraction layer
- OS provides programmers with *system libraries* to perform OS-level operations, e.g. using network connections, allocating memory, managing files, using peripherals (bluetooth, USB, etc.)

The role of the OS

- "Insulates" applications running on a computer, both from the *hardware* of that computer and also *from eachother*.
- "Insulates" the *user* from the complex inner workings of the computer.
- OS is itself a system! Components include:
  the *kernel*, the network stack, the filesystem, a hardware abstraction layer
- OS provides programmers with *system libraries* to perform OS-level operations, e.g. using network connections, allocating memory, managing files, using peripherals (bluetooth, USB, etc.)
- Primary function of the OS: managing the computer's *resources*.

Where do operating systems come from?

Whirlwind of CS history
1940s to 1980s

1940s

"Computer" is a job title. People – mostly women! – do calculations by hand.

1940s

"Computer" is a job title. People – mostly women! – do calculations by hand.

The first *electronic* computers are built in this decade. Women reprogrammed these *manually* by *rewiring* them. This could take weeks.

1940s

"Computer" is a job title. People – mostly women! – do calculations by hand.

The first *electronic* computers are built in this decade. Women reprogrammed these *manually* by *rewiring* them. This could take weeks.

Important invention in 1947: the transistor!

1950s

- An electronic computer, such as the UNIVAC or IBM 701, takes up an entire soccer-field-sized *room.*

1950s

- An electronic computer, such as the UNIVAC or IBM 701, takes up an entire soccer-field-sized *room.*
- The role of human computers shifts from doing calculations to operating these machines.

1950s

- An electronic computer, such as the UNIVAC or IBM 701, takes up an entire soccer-field-sized *room.*
- The role of human computers shifts from doing calculations to operating these machines.
- Although the transistor is invented, these machines use *vacuum tubes.*

1950s

- An electronic computer, such as the UNIVAC or IBM 701, takes up an entire soccer-field-sized *room.*
- The role of human computers shifts from doing calculations to operating these machines.
- Although the transistor is invented, these machines use *vacuum tubes.*
- It is still mostly women programming these, using punched cards.

1950s

- An electronic computer, such as the UNIVAC or IBM 701, takes up an entire soccer-field-sized *room.*
- The role of human computers shifts from doing calculations to operating these machines.
- Although the transistor is invented, these machines use *vacuum tubes.*
- It is still mostly women programming these, using punched cards.
- These do not really have OSes, but rather are *batch processing systems.*

1950s

- An electronic computer, such as the UNIVAC or IBM 701, takes up an entire soccer-field-sized *room.*
- The role of human computers shifts from doing calculations to operating these machines.
- Although the transistor is invented, these machines use *vacuum tubes.*
- It is still mostly women programming these, using punched cards.
- These do not really have OSes, but rather are *batch processing systems.*

https://www.youtube.com/watch?v=fsdLxarwmTk

1960s: the first operating systems

- MIT Compatible Time-Sharing System (CTSS): Multiple simultaneous users on the same system!

1960s: the first operating systems

- MIT Compatible Time-Sharing System (CTSS): Multiple simultaneous users on the same system!
- Multics project (mid 1960s) is a flop: too complex! But introduced many key ideas, such as hierarchical file systems (more on that soon)

1960s: the first operating systems

- MIT Compatible Time-Sharing System (CTSS): Multiple simultaneous users on the same system!
- Multics project (mid 1960s) is a flop: too complex! But introduced many key ideas, such as hierarchical file systems (more on that soon)
- Unix (1969) takes the keys ideas from Multics, simplified things a lot.

1970s and 1980s

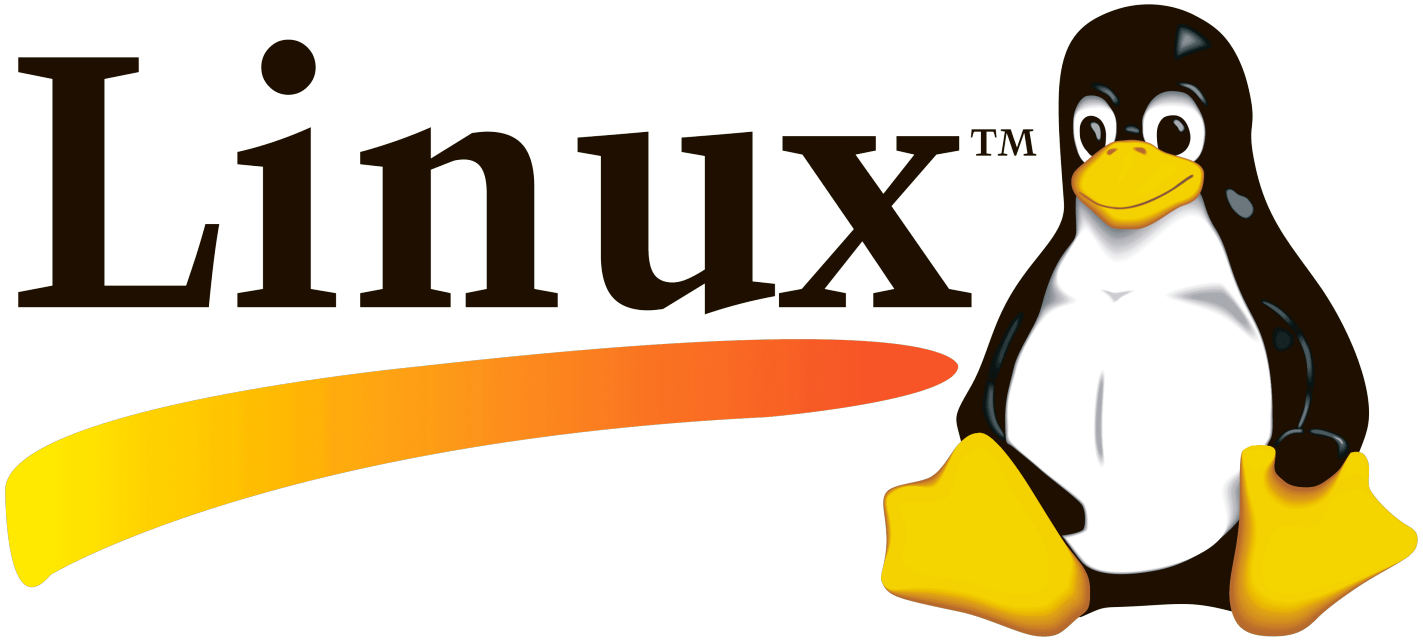
- Computers get smaller. At first, cabinet-sized, then desktop-sized.
- The *personal computer* becomes a thing.
- Proliferation of OSes: CP/M, Macintosh (1984), MS-DOS.
- Richard Stallman creates the GNU (GNU's Not Unix) project in 1983.

1991: the year Linux is born

```
From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)
Newsgroups: comp.os.minix
Subject: What would you like to see most in minix?
Summary: small poll for my new operating system
Message-ID:
Date: 25 Aug 91 20:57:08 GMT
Organization: University of Helsinki


Hello everybody out there using minix -

I'm doing a (free) operating system (just a hobby, won't be big and
professional like gnu) for 386(486) AT clones.  This has been brewing
since april, and is starting to get ready.  I'd like any feedback on
things people like/dislike in minix, as my OS resembles it somewhat
(same physical layout of the file-system (due to practical reasons)
among other things).

I've currently ported bash(1.08) and gcc(1.40), and things seem to work.
This implies that I'll get something practical within a few months, and
I'd like to know what features most people would want.  Any suggestions
are welcome, but I won't promise I'll implement them :-)

              Linus (torvalds@kruuna.helsinki.fi)

PS.  Yes - it's free of any minix code, and it has a multi-threaded fs.
It is NOT protable (uses 386 task switching etc), and it probably never
will support anything other than AT-harddisks, as that's all I have :-(.
```

Judging from the post, 0.01 wasn't actually out yet, but it's close. I'd guess the first version went out in the middle of September -91. I got some responses to this (most by mail, which I haven't saved), and I even got a few mails asking to be beta-testers for linux. After that just a few general answers to quesions on the net:

What's so special about Linux?



Free and open-source software

- Anyone can download and look at the source code of Linux.
- Anyone is welcome to make contributions to the code of Linux.
- Essentially developed by volunteers for the last 30 years!

Linux is just the *kernel*

- Recall: kernel is just the core of the OS.

Linux is just the *kernel*

- Recall: kernel is just the core of the OS.
- A full OS needs more than that.

Linux is just the *kernel*

- Recall: kernel is just the core of the OS.
- A full OS needs more than that.
- Many OSes are built using the Linux kernel:

Linux is just the *kernel*

- Recall: kernel is just the core of the OS.
- A full OS needs more than that.
- Many OSes are built using the Linux kernel:
  - The many so-called "Linux distributions" often include many programs from the GNU project: Ubuntu, Debian, CentOS, Red Hat Enterprise Linux, Arch Linux, Kali Linux, and many, many more.

Linux is just the *kernel*

- Recall: kernel is just the core of the OS.
- A full OS needs more than that.
- Many OSes are built using the Linux kernel:
  - The many so-called "Linux distributions" often include many programs from the GNU project: Ubuntu, Debian, CentOS, Red Hat Enterprise Linux, Arch Linux, Kali Linux, and many, many more.
  - There are also several non-GNU Linux distributions, e.g. Alpine Linux.

Linux is just the *kernel*

- Recall: kernel is just the core of the OS.
- A full OS needs more than that.
- Many OSes are built using the Linux kernel:
  - The many so-called "Linux distributions" often include many programs from the GNU project: Ubuntu, Debian, CentOS, Red Hat Enterprise Linux, Arch Linux, Kali Linux, and many, many more.
  - There are also several non-GNU Linux distributions, e.g. Alpine Linux.
  - Android and ChromeOS use the Linux kernel.

Monolithic kernel

- Kernel performs core OS functions, but what counts as "core"?

Monolithic kernel

- Kernel performs core OS functions, but what counts as "core"?
- In a monolithic kernel, *a lot:* networking, filesystem, user accounts / permissions, memory management, device drivers, and even more.

Monolithic kernel

- Kernel performs core OS functions, but what counts as "core"?
- In a monolithic kernel, *a lot:* networking, filesystem, user accounts / permissions, memory management, device drivers, and even more.
- This explains why Linux is a popular kernel for building many OSes out there.

Primarily used on servers

The vast, vast majority of web servers run Linux.

Command-line first

Notice there were no screens attached to those servers?

Command-line first

Notice there were no screens attached to those servers?

```
jerrin@teach-node-01 ~
  % ls                                                                  !1015
a.out              comp-273.md  fall2022         loc
bin                Desktop      fall2023.tar.gz  office365
comp-206           dotfiles     hw1              submissions
comp-206-grading   Downloads    hw5-dist.tar.gz  whos-online

jerrin@teach-node-01 ~
  % w                                                                   !1016
 09:46:04 up 11 days,  5:44,  1 user,  load average: 0.08, 0.13, 0.10
USER     TTY      FROM             LOGIN@   IDLE   JCPU   PCPU WHAT
jerrin   pts/0    132.206.3.222    09:45    4.00s  0.59s  0.02s w

jerrin@teach-node-01 ~
  % ps                                                                  !1017
    PID TTY          TIME CMD
1610081 pts/0    00:00:00 zsh
1610300 pts/0    00:00:00 ps

jerrin@teach-node-01 ~
  %                                                                     !1018
```

Getting Started with Mimi

- You should **already** have made an SOCS account
- If not, go to accounts.cs.mcgill.ca/account/ **right now** and do it!
- This should have assigned you a username and password
  - username format: first initial, fragment of last name
  - password format: impossible to remember! (Will fix.)

Connect to Mimi with `ssh` (1)

- Start a terminal
- Windows users:
  - Open windows menu and search for `cmd`
  - (Or use winkey+R, then enter `cmd`)
- Mac users:
  - Go to applications and search for Terminal

Connect to Mimi with `ssh` (1)

- Start a terminal
- Windows users:
  - Open windows menu and search for `cmd`
  - (Or use winkey+R, then enter `cmd`)
- Mac users:
  - Go to applications and search for Terminal
- Check for SSH presence: `ssh -V`
- If not present, something is very weird...
  - On windows, see Windows SSH tutorial

Connect to Mimi with `ssh` (2)

- Now you can connect. Run the following command:
- `ssh [user]@mimi.cs.mcgill.ca`
  - **Replace [user] with your SoCS username.**
  - ex for me: `ssh mkopin@mimi.cs.mcgill.ca`
- You will be prompted for a password. Enter the password you were given.
  - The password will **not appear as you type it**. This is by design for security. Copy+paste still works or you can type it manually.

Connect to Mimi with `ssh` (2)

- Now you can connect. Run the following command:
- `ssh [user]@mimi.cs.mcgill.ca`
  - **Replace [user] with your SoCS username.**
  - ex for me: `ssh mkopin@mimi.cs.mcgill.ca`
- You will be prompted for a password. Enter the password you were given.
  - The password will **not appear as you type it**. This is by design for security. Copy+paste still works or you can type it manually.
- Once connected, you can change your password:
- `passwd`
- Change to something you will remember.
  - Must use lower/upper case and at least one number/symbol!

Do Not Log In With Your Email

- **Do not** log in to Mimi with your `first.last` McGill account
- It might successfully connect you!
- But you will be logged in as a generic "campus user," not an SoCS student
- Many things you need for this course will not work correctly as campus user
- So... only log in with your given SoCS username
- **You have been warned!**

Demo:
Using the command-line
Paths

Exercise (if time / on your own)

- Create a `comp206/cmdline-exercise` directory
- Navigate into that directory
- Within it, create two new directories
- Create two files in each directory **without** entering them
- Create one new file in the current directory
- Run the command `tree` and describe the output
- You created 5 files: delete any two of them using **absolute** paths
- What do you think will happen if you run `tree` again? Discuss...
- ...then do it